

UADE

Trabajo Práctico Obligatorio de Programación Orientada a Objetos

Propuesta Elegida : C

Integrantes :

- Eric Nahuel Song
- Ian Lucas Hyun Chae

Etapa 1

1. Análisis del enunciado

a. Actores del Sistema

- Operador de Partido:Persona encargada de registrar los eventos en vivo de un partido en juego y cargar eventos de anulación cuando ocurra un error.

- Administrador del Torneo: Encargado de gestionar torneos, clubes, planteles, estadios, árbitros, sponsors, generar fixtures y programar partidos.

- Analista Estadístico: Usuario que solicita las tablas de posiciones, goleadores, valla menos vencida y requiere el cálculo de proyecciones (xG y probabilidades de clasificación).

- Suscriptor: Usuario que configura los clubes que sigue y los canales habilitados (app móvil, correo o SMS), y recibe notificaciones sobre goles, tarjetas rojas y finales de partido.

b. Requerimientos Funcionales (RF)

- RF1 (Administración General): Administrar torneos, clubes, estadios, jugadores, entrenadores y árbitros.

- RF2 (Inscripciones y Fixture): Inscribir clubes en un torneo con sus planteles y generar el fixture correspondiente.

- RF3 (Programación y Gestión de Partidos): Programar partidos asignando clubes (local y visitante), fecha/hora, estadio y terna arbitral, gestionar sus estados (Programado, En juego, Finalizado, Suspendido) y cerrar el partido.

- RF4 (Eventos en Vivo y Anulaciones): Registrar en tiempo real los eventos de un partido en juego (Gol, Remate, Tarjetas, Cambio y Falta), asociándolos al minuto y jugador, y permitir la corrección de errores mediante eventos de anulación vinculados al original.

- RF5 (Notificaciones Automáticas): Enviar notificaciones sobre goles, tarjetas rojas y finales de partido a los suscriptores según los clubes y canales configurados (App móvil, correo, SMS).

- RF6 (Contratos y Patrocinios): Gestionar contratos de jugadores (alta, transferencia y rescisión), conservando su historial, y gestionar los patrocinios de clubes y torneos.

- RF7 (Reportes y Estadísticas): Generar la tabla de posiciones, la tabla de goleadores y la valla menos vencida del torneo.

- RF8 (Proyecciones de Rendimiento): Calcular los goles esperados (xG) y la probabilidad de clasificación de un club mediante métodos de cálculo intercambiables.

c. Reglas de Negocio

1. Estado e Inscripción de Torneos: Un torneo solo puede pasar a estado "En curso" si cuenta con al menos 4 clubes inscriptos. Una vez en curso, no admite nuevas inscripciones.

2. Registro de Eventos: Solo pueden registrarse eventos cuando el partido se encuentra en estado "En juego". El jugador involucrado debe formar parte de la nómina de uno de los clubes del partido.

3. Cambios y Expulsiones: Cada club puede realizar como máximo 5 cambios por partido. Un jugador sustituido no puede reingresar y un jugador expulsado no puede ser reemplazado.

4. Sanciones por Tarjetas: La acumulación de 5 tarjetas amarillas en el torneo o una tarjeta roja directa suspende al jugador para la fecha siguiente, impidiendo su inclusión en la nómina.

5. Corrección de Eventos: Un evento registrado no se elimina. En caso de error, se registra un evento de anulación asociado al evento original.

6. Cálculo de Posiciones: La tabla de posiciones otorga 3 puntos por victoria, 1 por empate y 0 por derrota. Solo se consideran partidos finalizados.

7. Patrocinios: Un club no puede tener más de un sponsor principal vigente al mismo tiempo.

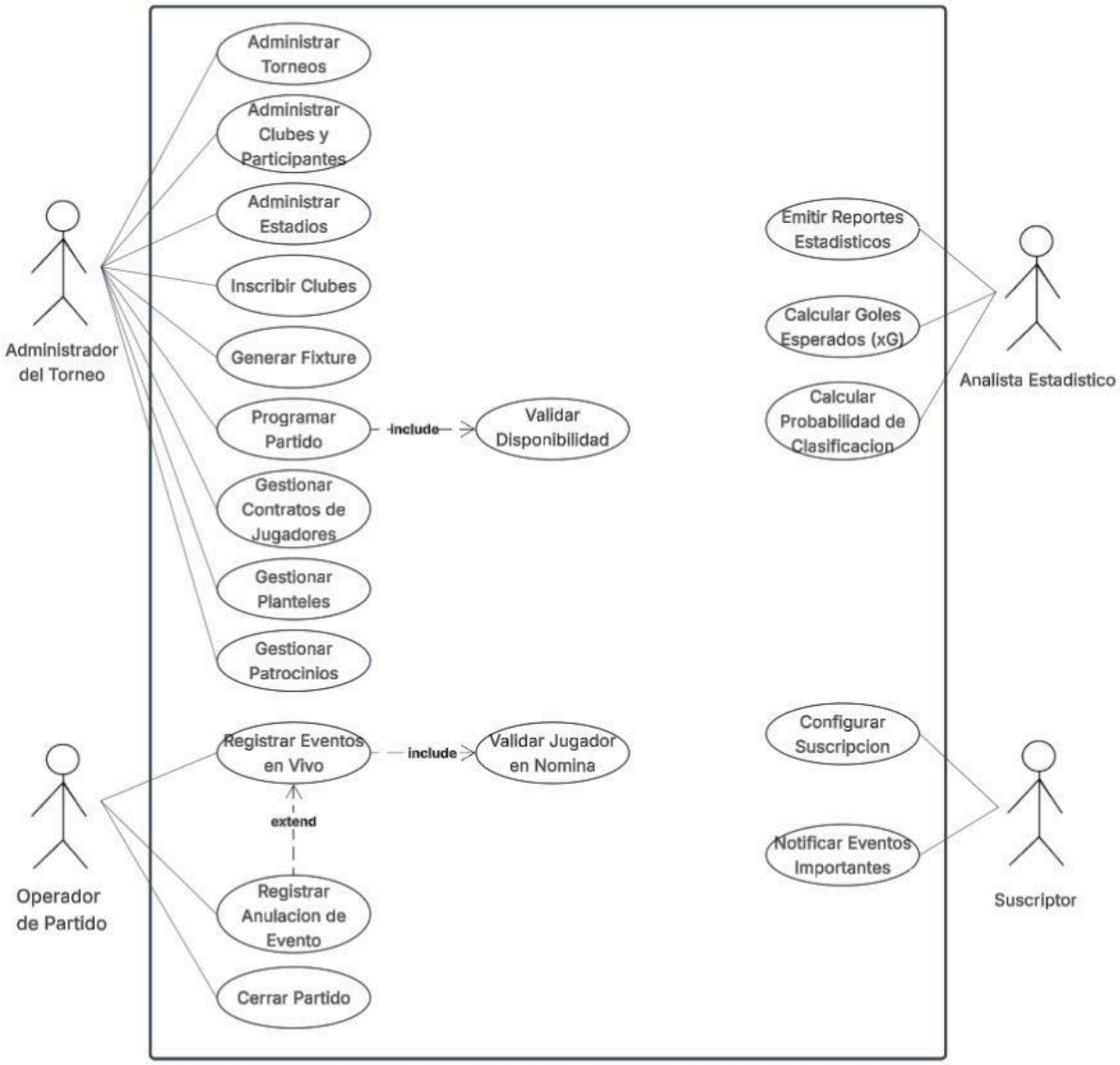
d. Supuestos

- SUPUESTO 1 (Identificación de Personas): Se considera que el DNI identifica de manera única a cada persona registrada en el sistema.

- SUPUESTO 2 (Nómina de Partido): Se considera que cada club dispone de una nómina de jugadores habilitados para cada partido, utilizada para validar el registro de eventos.

2. Diagrama de casos de uso

a. Diagrama de casos de uso



b. Descripción de casos de uso

1. *Módulo de Gestión y Logística (Actor: Administrador del Torneo)*

- CU01: Administrar Torneos
  - Actor principal: Administrador del Torneo.
  - Precondición: El usuario debe estar autenticado en el sistema con rol de administrador.

    - Flujo principal:
        1. El administrador selecciona la opción de gestionar torneos.
        2. El sistema muestra la lista de torneos existentes con sus estados (planificado, en curso, finalizado).
        3. El administrador selecciona crear un torneo nuevo o modificar uno existente.
        4. Ingresa los datos requeridos (nombre, fechas de inicio y fin).
        5. El sistema valida los datos y guarda los cambios en el torneo.
    - Flujos alternativos:
        - 3a. Cambio de estado: Si se intenta cambiar el estado de un torneo, el sistema valida que solo se pueda avanzar al estado siguiente y que no se inicie con menos de 4 clubes inscriptos.
- CU02: Administrar Clubes y Participantes
    - Actor principal: Administrador del Torneo.
    - Precondición: Estar acreditado en el sistema.
    - Flujo principal:
        1. El administrador accede al módulo de clubes y personas (jugadores, entrenadores, árbitros).
        2. Registra el alta, modificación o consulta de un club (nombre, fundación, sede social) o de una persona (DNI, nombre, apellido, fecha de nacimiento).
        3. El sistema almacena la información preservando los datos históricos de cada participante.
- CU03: Administrar Estadios
    - Actor principal: Administrador del Torneo.
    - Precondición: Estar acreditado en el sistema.
    - Flujo principal:
        1. El administrador ingresa los datos del estadio (nombre, capacidad, dirección y condiciones habituales de clima o césped).
        2. El sistema valida y registra el estadio para que quede disponible en la programación de partidos.
- CU04: Inscribir Clubes
    - Actor principal: Administrador del Torneo.
    - Precondición: El torneo debe encontrarse en estado *planificado*.
    - Flujo principal:
        1. El administrador selecciona un torneo y los clubes que participarán.
        2. El sistema valida que el torneo no esté en curso (no admite nuevas inscripciones) y que el club no esté ya inscripto en el mismo.
        3. El sistema confirma la inscripción del plantel asociado al club.
- CU05: Generar Fixture
    - Actor principal: Administrador del Torneo.
    - Precondición: El torneo debe contar con los clubes inscriptos requeridos (mínimo 4).
    - Flujo principal:

1. El administrador solicita la generación del calendario de enfrentamientos.
2. El sistema distribuye los partidos asegurando que las fechas estén dentro del período del torneo.

- CU06: Programar Partido
  - Actor principal: Administrador del Torneo.
  - Precondición: El fixture debe estar generado, y los estadios y árbitros dados de alta.
  - Flujo principal:
    1. El administrador selecciona un partido del fixture y asigna la fecha, hora, estadio y terna arbitral (un árbitro principal y dos asistentes).
    2. El sistema ejecuta el caso de uso incluido **Validar Disponibilidad**:
       - Comprueba que el estadio no aloje dos partidos con menos de 3 horas de diferencia.
       - Verifica que ningún club juegue con menos de 48 horas de diferencia.
       - Valida que los árbitros no dirijan a un club del que sean hinchas ni tengan otro partido ese día.
       - Controla que si es un torneo internacional, la terna sea de categoría internacional.
    3. El sistema guarda el partido en estado *programado*.
  - Flujos alternativos:
    1. 2a. Conflicto de disponibilidad: El sistema rechaza la programación y emite un mensaje de error detallando la regla incumplida.
- CU07: Gestionar Contratos de Jugadores
  - Actor principal: Administrador del Torneo.
  - Precondición: Clubes y jugadores cargados en el sistema.
  - Flujo principal:
    1. El administrador registra altas, transferencias o rescisiones de contratos.
    2. El sistema asocia al jugador con su club de destino y almacena las fechas de inicio y fin, conservando el historial de estadísticas previas.
- CU08: Gestionar Planteles
  - Actor principal: Administrador del Torneo.
  - Precondición: Club inscripto en el torneo.
  - Flujo principal:
    1. El administrador configura la lista de jugadores que componen el plantel para un torneo.
    2. El sistema valida que el plantel tenga entre 18 y 30 jugadores y que un jugador pertenezca a un solo club por vez en el mismo torneo.
- CU09: Gestionar Patrocinios
  - Actor principal: Administrador del Torneo.
  - Precondición: Existencia de sponsors y entidades (clubes o torneos).
  - Flujo principal:
    1. El administrador vincula un sponsor (nombre, marca, logo) a un club o torneo indicando monto, fecha de inicio y fin.

2. El sistema valida que no exista otro patrocinio vigente solapado y que el club no posea más de un sponsor principal vigente de manera simultánea.

2. *Módulo de Operación de Partidos (Actor: Operador de Partido)*

- CU10: Registrar Eventos en Vivo
  - Actor principal: Operador de Partido.
  - Precondición: El partido debe estar en estado *en juego*.
  - Flujo principal:
    1. El operador selecciona el tipo de evento ocurrido (gol, remate, tarjeta amarilla, tarjeta roja, cambio o falta).
    2. Ingresa el minuto del partido (de 0 a 90, o adicional hasta 120) y selecciona al jugador involucrado.
    3. El sistema ejecuta el caso de uso incluido **Validar Jugador en Nómina**: comprueba que el jugador esté convocado en la planilla del partido y que no haya sido sustituido previamente.
    4. El sistema solicita los atributos específicos (ej. tipo de gol, color de tarjeta, zona de remate).
    5. El sistema actualiza el marcador, las estadísticas y evalúa sanciones acumuladas.
    6. Si el evento es un gol o tarjeta roja, el sistema activa automáticamente la notificación a los suscriptores.
  - Flujos alternativos:
    1. Extensión (Registrar Anulación de Evento): Si el operador cometió un error en el registro, puede invocar este caso de uso para asociar un evento de anulación al original (los eventos no se eliminan físicamente).
    2. Acumulación de tarjetas: Si el jugador recibe su segunda tarjeta amarilla en el partido, el sistema genera de forma automática la tarjeta roja y expulsa al jugador.
- CU11: Cerrar Partido
  - Actor principal: Operador de Partido.
  - Precondición: El partido debe encontrarse finalizado en el campo de juego.
  - Flujo principal:
    1. El operador ingresa la orden de cierre del partido.
    2. El sistema cambia el estado del partido a *finalizado*, consolida el marcador definitivo y habilita los datos para las tablas de posiciones y estadísticas.

3. *Módulo de Analítica y Reportes (Actor: Analista Estadístico)*

- CU12: Emitir Reportes Estadísticos
  - Actor principal: Analista Estadístico.
  - Precondición: Existencia de partidos finalizados en el torneo.
  - Flujo principal:

1. El analista solicita un reporte (tabla de posiciones, goleadores o valla menos vencida).
2. El sistema calcula los puntos (Victoria: 3, Empate: 1, Derrota: 0) aplicando las reglas de desempate (diferencia de gol, goles a favor, resultado entre sí), identifica al goleador por goles a favor y a la valla menos vencida por menor promedio de goles recibidos.
3. El sistema muestra el reporte en pantalla.

- CU13: Calcular Goles Esperados (xG)
  - Actor principal: Analista Estadístico.
  - Precondición: Existencia de registros de remates en los partidos.
  - Flujo principal:
    1. El analista solicita el cálculo de xG para un jugador o club en un partido o torneo.
    2. El sistema evalúa la zona de cada remate aplicando los valores definidos (área chica: 0,40; área grande: 0,15; fuera del área: 0,04; penal: 0,76) y totaliza la métrica solicitada.
- CU14: Calcular Probabilidad de Clasificación
  - Actor principal: Analista Estadístico.
  - Precondición: Torneo en curso con datos de partidos disputados.
  - Flujo principal:
    1. El analista solicita el cálculo de probabilidades de clasificación.
    2. El sistema permite seleccionar el método de cálculo (proyección por promedio de puntos históricos o simulación por xG de partidos restantes).
    3. El sistema procesa los datos y presenta los porcentajes estimados.

4. *Módulo de Notificaciones (Actor: Suscriptor)*

- CU15: Configurar Suscripción
  - Actor principal: Suscriptor.
  - Precondición: Ninguna.
  - Flujo principal:
    1. El usuario accede a la plataforma de notificaciones y registra sus datos personales.
    2. Selecciona qué clubes desea seguir y qué canales de comunicación prefiere habilitar (aplicación móvil, correo electrónico o SMS).
    3. El sistema almacena las preferencias de suscripción.
- CU16: Notificar Eventos Importantes
  - Actor principal: Suscriptor.
  - Precondición: Ocurrencia de un evento crítico (gol, tarjeta roja o final de partido) durante un encuentro en juego.
  - Flujo principal:
    1. El sistema detecta el evento y recupera los suscriptores interesados en los clubes involucrados.

2. Envía la notificación en tiempo real a través de los canales configurados (App, Correo, SMS).

- Flujos alternativos:
    1. Falla de un canal: Si un canal de comunicación experimenta un fallo, el sistema lo registra internamente pero continúa el envío por los demás canales sin interrumpir la carga ni la operación del partido.

3. Diagrama de clases inicial

a. Identificación de clases

- *Enumeraciones (<<enumeration>>)*

  - **EstadoTorneo**: PLANIFICADO, EN_CURSO, FINALIZADO
  - **EstadoPartido**: PROGRAMADO, EN_JUEGO, FINALIZADO, SUSPENDIDO
  - **CategoriaArbitro**: NACIONAL, INTERNACIONAL
  - **ColorTarjeta**: AMARILLA, ROJA
  - **EspecialidadEntrenador**: DIRECTOR_TECNICO, PREPARADOR_FISICO, AYUDANTE_CAMPO
  - **TipoGol**: A_FAVOR, EN_CONTRA, PENAL
  - **ZonaRemate**: AREA_CHICA, AREA_GRANDE, FUERA_DEL_AREA, PENAL

- *Herencia*

  - **Personas**
    - Persona *(Clase Abstracta)*
      - Atributos: -dni: String, -nombre: String, -apellido: String, -fechaNacimiento: LocalDate
      - Métodos: +getDatosPersonales(): String
      - **Jugador** *(extiende de Persona)*
        - Atributos: -posicion: String, -numeroCamiseta: int, -valorMercado: double, -partidosJugados: int, -golesAcumulados: int, -tarjetasAcumuladas: int
        -
        - Métodos: +obtenerEstadisticas(): void, +incrementarTarjetasAmarillas(): void, +getTarjetasAcumuladas(): int, +expulsar(): void
      - **Entrenador** *(extiende de Persona)*
        - Atributos: -especialidad: EspecialidadEntrenador, -licencia: String
      - **Arbitro** *(extiende de Persona)*
        - Atributos: -colegioArbitral: String, -categoria: CategoriaArbitro

- *Clases Principales de Dominio*

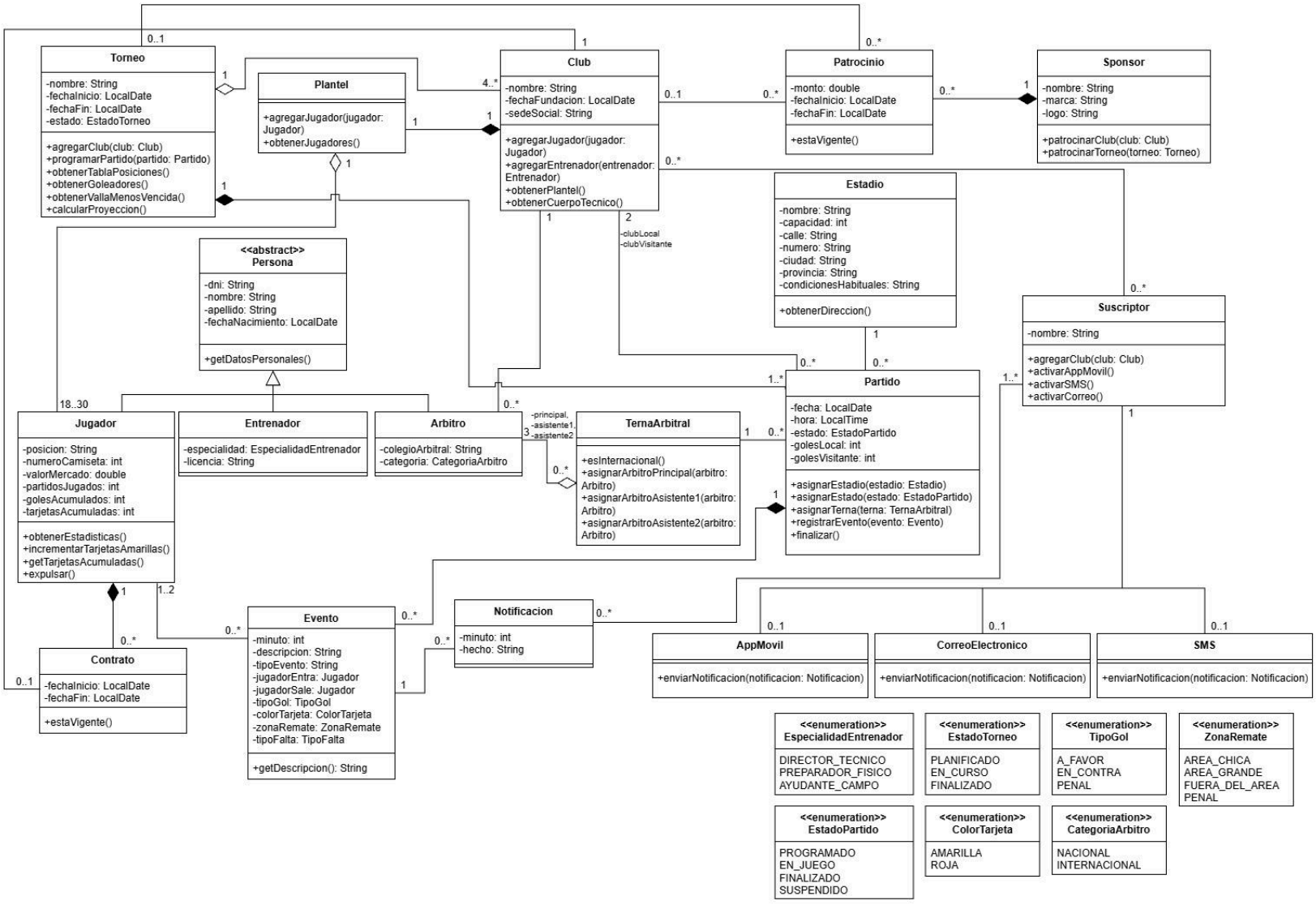
  - Torneo

    - Atributos: -nombre: String, -fechaInicio: LocalDate, -fechaFin: LocalDate, -estado: EstadoTorneo

    - Métodos: +agregarClub(club: Club): void, +programarPartido(partido: Partido): void, +obtenerTablaPosiciones(): void, +obtenerGoleadores(): void, +obtenerVallaMenosVencida(): void, +calcularProyeccion(): void
- Club
    - Atributos: -nombre: String, -fechaFundacion: LocalDate, -sedeSocial: String
    - Métodos: +agregarJugador(jugador: Jugador): void, +agregarEntrenador(entrenador: Entrenador): void, +obtenerPlantel(): void, +obtenerCuerpoTecnico(): void
- Plantel
    - Métodos: +agregarJugador(jugador: Jugador): void, +obtenerJugadores(): void
- Partido
    - Atributos: -fecha: LocalDate, -hora: LocalTime, -estado: EstadoPartido, -golesLocal: int, -golesVisitante: int
    - Métodos: +asignarEstadio(estadio: Estadio): void, +asignarTerna(terna: TernaArbitral): void, +registrarEvento(evento: Evento): void, +finalizar(): void
- Estadio
    - Atributos: -nombre: String, -capacidad: int, -calle: String, -numero: String, -ciudad: String, -provincia: String, -condicionesHabituales: String
    - Métodos: +obtenerDireccion(): String
- TernaArbitral
    - Métodos: +esInternacional(): boolean, +asignarArbitroPrincipal(arbitro: Arbitro): void, +asignarArbitroAsistente1(arbitro: Arbitro): void, +asignarArbitroAsistente2(arbitro: Arbitro): void
- Contrato
    - Atributos: -fechaInicio: LocalDate, -fechaFin: LocalDate
    - Métodos: +estaVigente(): boolean
- Sponsor
    - Atributos: -nombre: String, -marca: String, -logo: String
    - Métodos: +patrocinarClub(club: Club): void, +patrocinarTorneo(torneo: Torneo): void
- Patrocinio
    - Atributos: -monto: double, -fechaInicio: LocalDate, -fechaFin: LocalDate
    - Métodos: +estaVigente(): boolean
- Notificacion
    - Atributos: -minuto: int, -hecho: String
    - Métodos: +enviarNotificacion(): void
- Eventos
    - Atributos: -minuto: int, -descripcion: String, -tipoEvento: String, -jugadorEntra: Jugador, -jugadorSale: Jugador, -tipoGol: TipoGol, -colorTarjeta: ColorTarjeta, -zonaRemate: ZonaRemate, -tipoFalta: TipoFalta
    - Métodos: +getDescripcion(): String
- Suscriptor
    - Atributos: -nombre: String

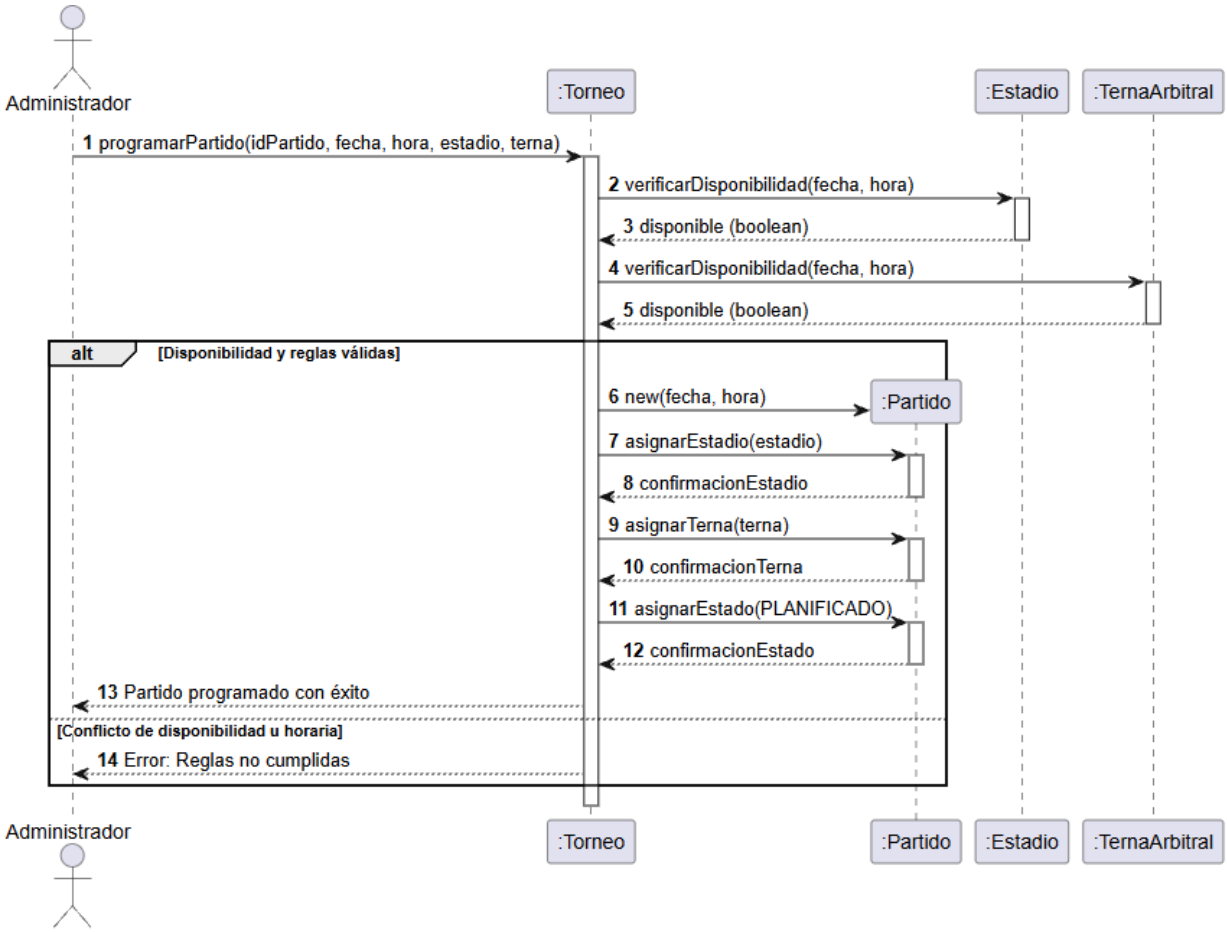
- Métodos: +agregarClub(club: Club): void, +activarAppMovil(): void, +activarSMS(): void, +activarCorreo(): void
- AppMovil / CorreoElectronico / SMS
  - Métodos: +enviarNotificacion(notificacion: Notificacion): void

b. Diagrama de clases Inicial

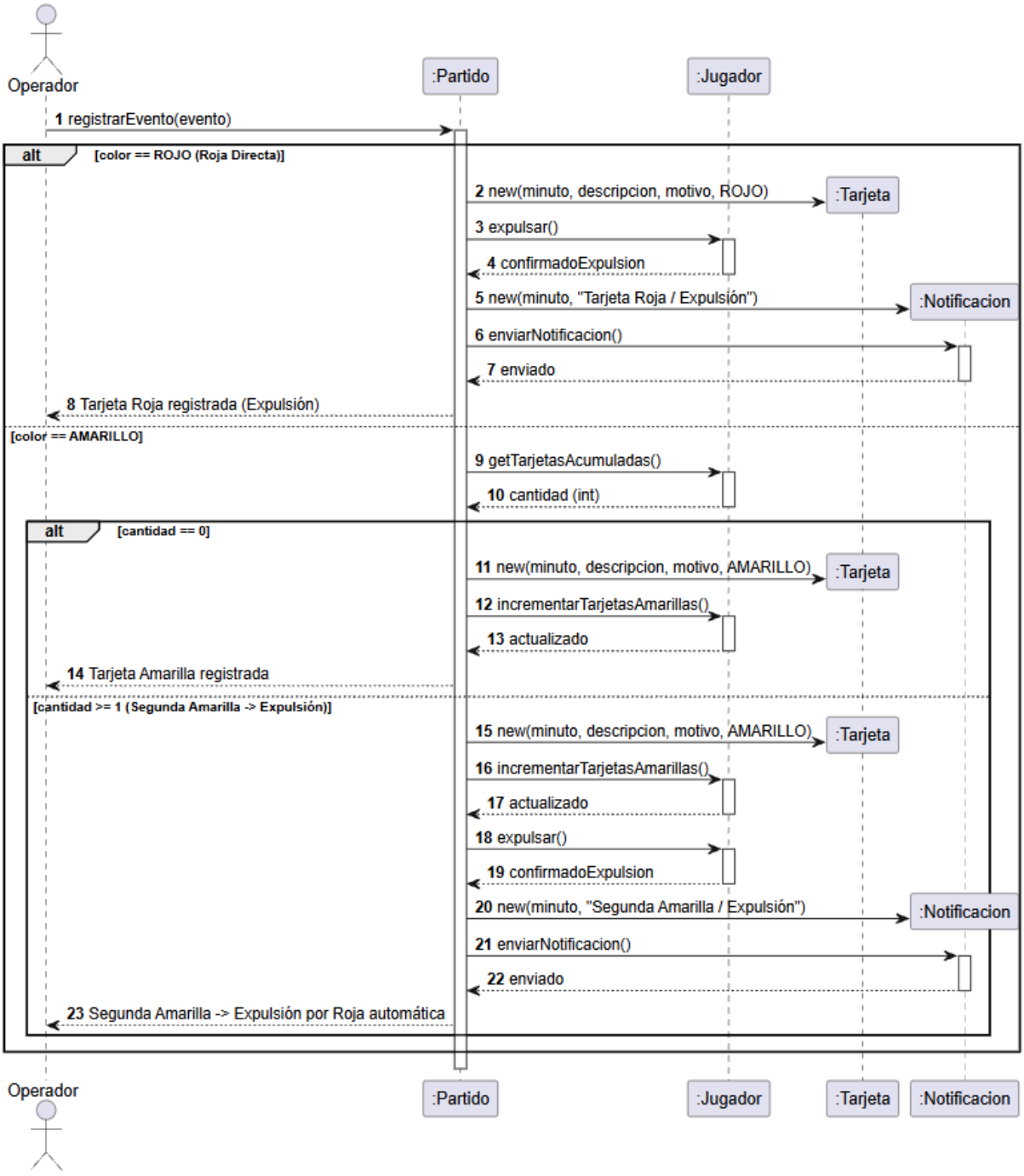


4. Diagrama de secuencias

a. programarPartido()



b. registrarEvento()



5. Aplicación de Patrones

a. Aplicación de Principios SOLID

- *Principio Abierto/Cerrado (OCP)*

  - **Problema detectado:** El enunciado establece que el sistema debe ofrecer distintos métodos para calcular proyecciones (promedios históricos, simulaciones xG) y debe permitir incorporar nuevas fórmulas en el futuro sin modificar los reportes. En nuestro diagrama inicial, la lógica matemática estaba centralizada dentro de la clase **Torneo** a través del método **calcularProyeccion().** Si dejábamos esto así, cada vez que nos pidieran un cálculo nuevo íbamos a tener que modificar el código central del dominio.
  - **Decisión tomada:** Decidimos abstraer el algoritmo de cálculo utilizando una interfaz, aplicando el concepto del patrón Estrategia.
  - **Clases donde se refleja:** Se creó la interfaz **EstrategiaProyeccion** y las clases que la implementan (**ProyeccionPromedioHistorico** y **ProyeccionSimulacionXG**).
  - **Ventaja:** El diseño queda abierto a la extensión (podemos sumar nuevas estrategias de cálculo simplemente agregando clases) pero cerrado a la modificación (el código base que pide la proyección no cambia).

- *Principio de Responsabilidad Única (SRP)*

  - **Problema detectado:** En el diseño inicial, la clase **Torneo** estaba sobrecargada de responsabilidades. Por un lado, gestionaba la lógica natural del campeonato (fechas, inscripciones de clubes, estados) y, por otro, se encargaba de procesar los datos para armar la tabla de posiciones, buscar a los goleadores y sacar la valla menos vencida. Tenía múltiples motivos para cambiar.
  - **Decisión tomada:** Extraer toda la lógica de cálculo estadístico y reportes, delegándola a una clase dedicada exclusivamente a ese fin.
  - **Clases donde se refleja:** La clase **Torneo** (que ahora solo maneja el campeonato) y la nueva clase **GeneradorReportes**.
  - **Ventaja:** Aumentamos la cohesión del código. Esto facilita el mantenimiento y el testeo unitario, ya que un cambio en el formato visual o lógico de la tabla de posiciones no impacta en absoluto en el funcionamiento interno del torneo.

- *Principio de Inversión de Dependencias (DIP)*

  - **Problema detectado:** El sistema notifica eventos a los suscriptores y tiene que estar preparado para sumar nuevos canales (como WhatsApp o Telegram). En nuestro modelo inicial, el **Suscriptor** tenía métodos específicos (**activarAppMovil**, **activarSMS**) y se relacionaba mediante asociaciones directas a las clases concretas de cada tecnología. Esto generaba un acoplamiento rígido a herramientas de bajo nivel.
  - **Decisión tomada:** Invertimos la dependencia. Modificamos el diseño para que el suscriptor dependa únicamente de una abstracción (una interfaz) y no de las clases concretas de cada medio de comunicación.
  - **Clases donde se refleja:** La clase **Suscriptor**, la interfaz **CanalNotificacion**, y las implementaciones (**AppMovil**, **SMS**, **CorreoElectronico**).
  - **Ventaja:** Podemos incorporar cualquier canal nuevo de mensajería creando una sola clase que respete el contrato de la interfaz, sin necesidad de tocar el código de los suscriptores.

b. Aplicación de Patrones GRASP

- *Fabricación Pura*

  - **Problema detectado:** Teníamos operaciones con mucha carga de procesamiento de datos, como los cálculos de "Goles Esperados (xG)" y el cruce de datos para los reportes estadísticos. Asignarle esta responsabilidad a clases como **Torneo** o **Partido** ensuciaba su propósito principal, ya que un "generador de estadísticas" no es un concepto que exista físicamente en el mundo real del fútbol.

  - **Decisión tomada:** Creamos una clase artificial que no representa un ente del dominio real, con el único objetivo de agrupar estas operaciones de análisis de forma cohesiva.
  - **Clases donde se refleja:** La clase inventada **GeneradorReportes** (y el motor estadístico).
  - **Ventaja:** Evitamos contaminar las clases principales del dominio con cálculos matemáticos complejos, manteniendo su alta cohesión y logrando que los algoritmos estadísticos queden en un módulo reutilizable.

- *Controlador*

  - **Problema detectado:** Al analizar los casos de uso, notamos que actores externos (como el Operador de Partido o el Administrador) enviaban comandos de negocio (por ejemplo, registrar un evento en vivo) interactuando directamente con las clases del modelo, como **Partido**. Esto acoplaba la capa de interfaz de usuario con la lógica del negocio.

- **Decisión tomada:** Introducir una clase controladora que actúa como fachada. Este controlador recibe todos los eventos de la interfaz y los reparte a los objetos internos que correspondan.
- **Clases donde se refleja:** La nueva clase **ControladorSistemaFutbol**.
- **Ventaja:** Desacopla completamente la capa de presentación de la capa de dominio. Esto garantiza que el core de la aplicación funcione igual de bien tanto desde una interfaz gráfica como desde una terminal por consola.

- *Indirección*

- **Problema detectado:** En nuestro diagrama inicial, la clase **Notificacion** interactuaba directamente con el **Suscriptor** para entregarle el aviso. Esto significaba que el proceso de generación del evento estaba íntimamente ligado al proceso de ruteo y entrega de mensajes a los usuarios.
- **Decisión tomada:** Agregamos un objeto intermediario para manejar los envíos. Ahora, los eventos derivan en notificaciones que se envían a este gestor, y es el gestor quien se encarga de buscar a los usuarios y despachar.
- **Clases donde se refleja:** La inclusión de la clase **GestorNotificaciones** como puente entre los eventos del **Partido** y los **Suscriptores**.
- **Ventaja:** Bajamos el acoplamiento drásticamente. Si el servicio de envíos se cae, colapsa o demora, el partido puede seguir registrando sus eventos sin verse bloqueado.

- *Polimorfismo*

- **Problema detectado:** El sistema debe registrar muchos tipos de eventos (goles, tarjetas, cambios, faltas), y cada uno requiere su propio formato para mostrarse en el acta (el gol necesita saber si es de penal, la tarjeta necesita el color, el cambio necesita dos jugadores). En el diagrama inicial, agrupamos todo en una clase gigante llamada **Evento** con atributos mezclados. Para imprimir el detalle, íbamos a necesitar una estructura de **if/else** o **switch** inmensa consultando el atributo tipo.
- **Decisión tomada:** Separamos los tipos de eventos creando una jerarquía de herencia. Declaramos el método **getDescripcion()** en una superclase abstracta y dejamos que cada tipo de evento concreto lo resuelva a su manera.
- **Clases donde se refleja:** La clase abstracta **Evento** y sus clases hijas (**Gol**, **Tarjeta**, **Cambio**, **Remate**, **Falta**).
- **Ventaja:** Eliminamos la lógica condicional anidada y optimizamos la memoria (ya no hay atributos nulos sin usar). Para armar el acta, el sistema simplemente recorre la lista llamando a **getDescripcion()**, y Java se encarga dinámicamente de ejecutar el comportamiento de la clase que corresponda.

6. Diagrama de clases final

